

ComicCraft – AI Comic Story Creator Using Gemini Models

3. Project Design Phase

Designing the Application Architecture, Modules & User Flow

1. Design Objective

The design phase organizes ComicCraft into connected frontend, backend, and AI-integration components. Each part is assigned a specific responsibility so that the complete comic-generation workflow can operate in a structured manner.

2. High-Level Architecture

Frontend: HTML, CSS, and Jinja2 templates are used to collect user information and display the generated results. **Backend:** FastAPI receives requests, manages routes, coordinates the processing steps, and prepares responses. **AI Integration:** Gemini APIs generate the story content, while Stable Diffusion is responsible for creating illustrations.

3. Frontend Design

The main interface is provided through **index.html**, which contains fields for the story prompt, main character name, setting, tone, and art style. The **comic_preview.html** page displays the generated panels in sequence, while **export_success.html** confirms successful comic export. The interface design includes a scenic background, centered form layout, readable typography, rounded components, shadows, and hover effects for buttons.

4. Backend Design

The primary routing logic is maintained in **routes.py**. It accepts both form and JSON requests, communicates with the generation modules, constructs the comic layout, and returns either a rendered web page or a JSON response. Error handling is supported through FastAPI's HTTPException and try-except mechanisms.

5. AI Module Design

gemini_flash.py contains **generate_outline()** for creating the structured five-panel outline. **gemini_pro.py** contains **generate_story()** for producing narration and dialogue. **image_generator.py** provides **generate_image()** for generating panel illustrations and stores the resulting images inside **static/panels**.

6. Layout and Export Design

layout_builder.py contains **build_comic_layout()**, which connects each generated image with its corresponding panel story. **exporters.py** provides **save_pdf()** to create the multi-page PDF containing panel images and narration. The exported files are stored in **static/exports**.

7. Route Design

The main application routes are **/** for the homepage, **/generate** for form-based comic generation, **/generate-comic/json** for JSON-based generation, **/export-success** for export confirmation, and **/test-image** for direct image-generation testing.

8. User Flow

The user first opens the homepage and enters the required preferences. After submitting the form, the backend generates the outline, story, and illustrations. The user then reviews the generated comic on the preview page and downloads the PDF. Finally, the application displays the export-success page.

9. Design Outcome

The design phase establishes the application's component boundaries, responsibilities of individual modules, page templates, routes, and overall user journey. These design decisions provide the structure needed for the development and integration stages.